

Flash Attention 4

→ Attention Kernel for B200s (sm-100 GPUs)

sm-xx → Refers to compute capability (cc)

X.Y → X → Major revision
Y → Minor revision

FA 1 → bf16 TFLOPS
~100

A100-40GB

FA 2 → ~200

A100-80GB
SXM

FA 3 → ~750

H100 SXM

FA 4 → ~1500

B200
30% MFO

(cudnn - Fmha is also petaFLOP scale)

On top of FA3, FA4 adds

→ more warp specialisation

→ Thread in a particular warp does a special task

→ 5 groups of tasks as compared to 2 in FA3

→ smarter, more efficient approach to numerical stability in softmax

→ 'online' softmax improvement

→ 'Software emulation of exponentiation' to relieve bottlenecks on the Special Func Units (SFU)

FAH on blackwell

→ does not use new FPH

→ does not use the new

2CTA matmul

→ ThunderKittens Blackwell
also does!

→ CTA → coop Thread
arrays

→ Kernel grid has many
little elements in it
usually need to program
just one at a time, but
in blackwell can program
2 - at - once

Flash Attention 1-3 : How did we
get here?

SM120 GPU (3090) have more
SFU bandwidth, so we don't need
the 'software softmax'.

FA1 applies tiling via on line
softmax algorithm

Break up ops onto some tiles
in memory

Key idea: Take chunks of K, Q, V ,
do compute, & then write
it to HBM.

Key parallelism strategy

Softmax looks trivial to parallelize

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Denom needs ~~sum~~ over all values.

Hard part of softmax: Numerically unstable!!

Safe softmax:

$$\sigma(z_i) = \frac{e^{B(z_i - m)}}{\sum_{j=1}^K e^{B(z_j - m)}}$$

$$m = \max(z)$$

exp of a large -ve number is close to 0.

Problem: $\max(z_i)$ is a reduction.

Naively, adds 2x overhead on memory reads.

Flash-attention trick: apply the above stabilization factor online.

New formula:

$$\sigma(x_i) = \frac{e^{\beta(z_i - m_j)}}{\sum_{j=1}^K e^{\beta(z_j - m_j)}}$$

$$\text{where } m_j = \max_{j' < j} z_{j'}$$

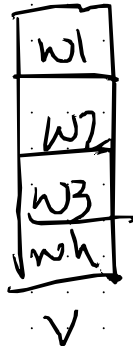
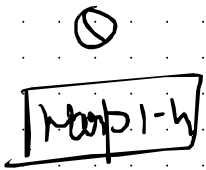
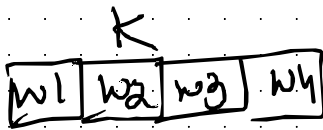
Also correct the past outputs:

$$\frac{\max z_j}{\max z_{j'}}$$

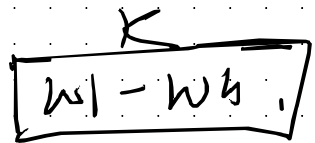
Flash Attention 2:

- Faster than FA1
- FA2 (~~transposes~~) the inner matmul
- Load Q tile, scan through keys to compute OK, & then scan values v to get final value.

FA1



FA2



→ In FA1, every tiles 1-4 were loaded together

→ In FA2, the output for a tile, only depends on block w)

→ Hence, this gives 'warp partitioning', NOT specialization.

→ Triton folks came up with this.

→ Higher MFO

FA3:

→ Hopper GPR

→ Adds 2 key things

Asynchrony

→ issue loads with async

TMA

→ given async matmul accums (MMA_{as}) with warpgroup mma

- Common in CPU programming
- Eg: Tell OS kernel to send some bytes on the n/w, & let me know when response is ready.
- Async await in Python / NodeJS
- Async TMA ⇒ special H/W for loading tensors
- Async Matmul ⇒ Kick off a matmul, & come back to collect it
- Do other stuff when tensor core is running

Warp Specialization

- 2nd big contribution
- In your running kernel (cooperative thread array (CTA)) some warps do A & some do B

→ Not a new idea

→ SINGLE 2012 paper → combination
reaction paper first introduced
it.

→ Dynamic register allocation:

Give less registers to producers,
and more to consumers.

This is useful since consumers
can use extra registers for
computations. Producers only
do tasks like telling TMA
to write something to shared
memory, & hence don't need
all the above.

→ Warp group → n contiguous warps
with an alignment of 128
on thread idx. They are

scheduled better for matmul
on Hopper.

Warpgroups are removed in
FAH, so forget about them.

Warp Specialization

→ Program: Copy single Q tile to SRAM
Copy all K tiles to SRAM
Copy all V tiles to SRAM
Compute output
Store

→ Above program is written for
a single thread

→ CUDA runs the same program for
all threads in a thread block
Also called cooperative thread array

→ we run a grid of such threadblocks to utilize all streaming multiprocessors

→ CUDA guarantees that all threads in a thread block will always access the same SM.

Fun trick:

→ take groups of 32 threads (warps) that execute together

→ we can identify which threads will fall into a warp before execution, since it just depends on thread index.

→ we can specialize threads in a warp & say that first n

threads will only do loading i.e.
copying $Q, K \& V$ tiles to SRAM,
next block of n will do compute
operations.

→ Deepseek was using warp
specialization to handle
different parts of communication
cross GPUs. Can do crazy
shit

Asynchrony within warpgroups - minimal
sm
overlap

→ Skipped since its important
only to FA3

Flash Attention 1

- More asynchrony & better softmax efficiency
- Asynchrony: more stages of pipelining, more complex coordination
 - Basically like writing your own async await event loop.
 - Need to ensure programs are correct.
- Padded warpgroups
- Efficient softmax:
 - Don't rescale every time the max changes: only rescale when we need to for numerically correct O/P

→ Run the exponentiation off of hardware acceleration in SFUs, & onto the CUDA cores SOMETIMES. Only done when the SFUs are a bottleneck.

→ 5 different specialised type of warps

→ Specialized warps also called warp scrolls.

→ 'Load' warp

→ Very similar to 'Producers' warp

→ Just produces Q, K, V tiles into memory

→ 'Epilogue' warp

→ Reverse of 'Load' warp

→ Takes the output tiles
& writes them out
to memory.

→ 'MMA' warps

→ Per tile, there is one
MMA for doing matmul
between O & K

→ One MMA for matmul of
attention scores with
values to produce outputs.

→ 'Softmax' warps

→ Calc softmax

→ 'Correction' warps

→ Correct softmax

} Per tile
there are
4 warps

(Softmax) warps

- Taking raw attention scores & apply softmax
- Signal to (correction) warps when its time to do the rescaling.

Blackwell also introduces Tensor Memory, Not related to TMA, & different from shared memory.

- output memory of tensor cores
- Any output produced by tensor cores is stored here
- want to store things you are accumulating over, which is what we do in attention scores calc.

→ similar to shared memory,
but only accessible to tensor
cores

→ Kinda looks like another
memory layer, after shared
memory.

All this async stuff, do we have to
continue to do it by hand, or can
compiler figure it out?

→ The rise of tile based
DSLs is the eventually make
programmers write tile level
logic, & let compiler figure
out stuff.

→ This is why Gerohot said
tile is the right abstraction

→ Thunder Kittens FTW !!

→ Primarily need all this async stuff to drive Tensor cores at MAX flops. Many kernels, that at their core are NOT matmuls, don't need this async stuff. Eg: Normalizations, non-linearities.

→ Tile based model is better for us, since only at the tile level do we know about specialisation. Threads in thread model have no notion of tiles, and therefore don't lend themselves to be good.

Software Exponentiation

- Need to do exponential in softmax
- This is not a natural. Can't use softmax for it.
- Usual way to do it is to use a PTX intrinsic function like $\exp_2$, & convert back to $\exp$ by doing $\log_2 2$
- SFUs have way less bandwidth than Tensor Cores
- There are not many of them, so can stall
- Already doing polynomial approx (mul & add) of 2^x .

→ CUDA cores can do mul & add

→ Also has some packed instructions to do stuff to gether, more B/w & better scheduling.

→ Why not just do it in CUDA cores?

Key change:

→ The softmax function can be approximated using a cubic polynomial.

→ Because of this polynomial, we no longer need to always use the PTX intrinsic function.

→ In the code implementation, we have 2 blocks.

→ First block is executed if we're not planning to use any new way of softmax calc.

→ If we plan to use new softmax we first check a variable `log-ent`, which tracks how many times we use the cubic approx method

→ we use the method only if the count is under a certain value.

→ If not, fallback to using PTX intrinsic method.

→ The cubic poly approx code is even using custom assembly code!

One trick for softmax

- Do scaling earlier
- We subtract the max value from the current before the softmax, to be numerically correct, & avoid overflow/underflow issues.
- Don't have to always subtract the max value to be numerically correct
- We just need to make sure the numbers don't get too big, when they are exponentiated
- Trick: Defer the subtraction until the value has gone up so much that it actually matters

- Correction involves coordination, where need to rescale the previous output values
- This is not very expensive, BUT coordination is always going to have a cost.
- Complex logic.
- Setting the threshold value correctly gets upto a 10x reduction in the no. of times correction needs to be done.
- Builds on a lot of existing OS kernels. Nvidia Cutlass 77 - blackwell - forha
has same warp rules, & ldx assignments.

Flash Attention 5+

- Async lawait movement in Rust is implemented as a state machine
- GPU programming for Nvidia needs a big update
- CuTile
 - Nvidia, Tile based programming
 - Program at a tile level
 - Steven Jones
 - Boyle
 - Doesn't go through PTX anymore!!!!
ooo
 - Gets compiled to tile IR, a new IR, which then gets reduced to SASS

→ Triton / GLOW

→ Open AI

→ LLVM IR lowering

→ But looks like Triton folks have given up on this approach.

→ Problems

→ Kernels are becoming too complex

→ Hardware is gaining more degrees of freedom.

→ Solution? New backend called GLOW

→ Allows optimal tensor layout assignment in registers & in memory.

→ warp specialization & loop scheduling

→ LLVM sync steps to square more loop stages

→ Given FMA reports about similar performance to cuDNN.

D128, Rps gives 1500 TFLOPS

→ Mojo - Modular

→ General purpose DSL

→ LLVM-IR lowering

Good time for HPC !!